

Servidor HTTP 1.1 Concorrente: Implementação e Avaliação do Produto Mínimo Viável 1

João Oliveira Duarte, Raul Etcheverry Reis

Engenharia de Computação (UNIPAMPA)

Bagé – RS – Brasil

Resumo

Este artigo descreve a primeira fase de desenvolvimento de um servidor web com suporte a Qualidade de Serviço (QoS). Neste Produto Mínimo Viável (MVP 1), foi implementado um servidor HTTP concorrente em linguagem C, operando em ambiente Linux com o uso da biblioteca Pthreads. A avaliação experimental, conduzida por meio de captura de tráfego com a ferramenta Wireshark, demonstrou o pleno suporte a conexões simultâneas e a manutenção de sessões persistentes em conformidade com o protocolo HTTP 1.1.

1. Introdução

A arquitetura cliente-servidor e os protocolos da camada de aplicação formam a base estrutural da internet moderna. O protocolo HTTP (*Hypertext Transfer Protocol*), em sua versão 1.1, introduziu melhorias significativas no desempenho das redes ao adotar conexões persistentes (*Keep-Alive*) por padrão, permitindo que múltiplos objetos de uma mesma página web sejam trafegados através de uma única conexão TCP.

O presente trabalho prático, desenvolvido para a disciplina de Fundamentos e Avaliação de Redes de Computadores, tem como objetivo geral a compreensão destes conceitos por meio da implementação prática de um servidor HTTP com controle de Qualidade de Serviço (QoS). Este relatório detalha a metodologia, a implementação e a avaliação da primeira entrega do projeto (MVP 1), focada estritamente na concorrência computacional e no suporte a requisições persistentes.

2. Desenvolvimento

A especificação do MVP 1 exige o desenvolvimento de um servidor capaz de tratar múltiplas conexões de clientes simultaneamente, operando sobre o protocolo HTTP 1.1. O sistema foi implementado em ambiente Linux, utilizando a linguagem C e a interface de *sockets* de Berkeley.

Para o tratamento da concorrência, adotou-se o modelo *multithreading* com a biblioteca POSIX Threads (Pthreads). O fluxo de execução do servidor obedece à seguinte lógica:

1. Inicialização: O servidor cria um *socket* TCP (IPv4), vincula-o à porta de escuta (*bind*) e aguarda conexões (*listen*).
2. Despacho de Conexões: Ao receber uma requisição, a chamada bloqueante *accept* retorna um novo descritor de arquivo. O servidor principal não processa os dados; em vez disso, aloca memória para este descritor e invoca *pthread_create* para delegar o atendimento a uma *thread* independente.
3. Persistência (HTTP 1.1): Dentro da *thread*, um laço de repetição (*while(1)*) é estabelecido. O servidor realiza a leitura do caminho do arquivo no cabeçalho GET, acessa o disco rígido, extrai o tamanho exato do arquivo para montar o cabeçalho Content-Length e transmite os *bytes* do arquivo (texto ou imagem) em blocos. A *thread* não encerra a conexão TCP após o envio, mantendo-a ativa para requisições subsequentes do mesmo cliente, encerrando o ciclo apenas em caso de falha de comunicação ou recebimento da instrução explícita *Connection: close*.

3. Avaliação Experimental

Para validar a corretude da implementação frente aos requisitos funcionais, o servidor foi executado em um ambiente de emulação via Windows Subsystem for Linux (WSL), escutando requisições na interface *localhost*. O teste consistiu no acesso simultâneo à página raiz do servidor a partir de múltiplas instâncias de navegadores web (Microsoft Edge), os quais requisitaram um arquivo HTML que referenciava múltiplas imagens de vários megabytes de tamanho.

A análise do comportamento da rede foi realizada utilizando a ferramenta Wireshark. Os pacotes capturados evidenciam dois comportamentos fundamentais do MVP 1:

- Tratamento Concorrente: A captura registrou processos distintos de estabelecimento de conexão (*Three-way Handshake*) partindo de portas de origem diferentes em janelas de tempo sobrepostas. A transmissão de segmentos de dados TCP sinalizados com a *flag* [PSH, ACK] ocorreu de forma intercalada entre estas portas, comprovando que as instâncias *Pthreads* responderam aos clientes simultaneamente e não em fila de espera.
- Conexões Persistentes: A análise do fluxo de uma mesma porta de origem atesta a ausência da *flag* de encerramento de conexão [FIN, ACK] após o tráfego do arquivo index.html. O cliente utilizou a mesma sessão TCP previamente estabelecida para solicitar os arquivos de imagem, comportamento característico do protocolo HTTP 1.1 implementado no servidor.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	74	40164 → 8080 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=462896686 TSecr=0 WS=128
2	0.000024	127.0.0.1	127.0.0.1	TCP	74	8080 → 40164 [ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=462896686 TSecr=462896686 WS=128
3	0.000046	127.0.0.1	127.0.0.1	TCP	66	40164 → 8080 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=462896686 TSecr=462896686
4	0.000261	127.0.0.1	127.0.0.1	HTTP	786	GET / HTTP/1.1
5	0.000270	127.0.0.1	127.0.0.1	TCP	66	8080 → 40164 [ACK] Seq=1 Ack=721 Win=64768 Len=0 TSval=462896686 TSecr=462896686
6	0.000308	127.0.0.1	127.0.0.1	TCP	74	40180 → 8080 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=462896686 TSecr=0 WS=128
7	0.000315	127.0.0.1	127.0.0.1	TCP	74	8080 → 40180 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=462896686 TSecr=462896686 WS=128
8	0.000322	127.0.0.1	127.0.0.1	TCP	66	40180 → 8080 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=462896686 TSecr=462896686
9	0.002030	127.0.0.1	127.0.0.1	TCP	170	8080 → 40164 [PSH, ACK] Seq=1 Ack=721 Win=64768 Len=104 TSval=462896688 TSecr=462896686 [TCP PDU reassembled in 11]
10	0.002071	127.0.0.1	127.0.0.1	TCP	66	40164 → 8080 [ACK] Seq=721 Ack=105 Win=65536 Len=0 TSval=462896688 TSecr=462896688
11	0.002288	127.0.0.1	127.0.0.1	HTTP	401	HTTP/1.1 200 OK (text/html)
12	0.002320	127.0.0.1	127.0.0.1	TCP	66	40164 → 8080 [ACK] Seq=721 Ack=440 Win=65280 Len=0 TSval=462896688 TSecr=462896688
13	0.002750	127.0.0.1	127.0.0.1	HTTP	712	GET /image1.jpg HTTP/1.1
14	0.034428	127.0.0.1	127.0.0.1	TCP	160	8080 → 40164 [PSH, ACK] Seq=440 Ack=1367 Win=64128 Len=94 TSval=462896720 TSecr=462896719
15	0.035272	127.0.0.1	127.0.0.1	TCP	32834	8080 → 40164 [PSH, ACK] Seq=534 Ack=1367 Win=64128 Len=32768 TSval=462896721 TSecr=462896719
16	0.035313	127.0.0.1	127.0.0.1	TCP	66	40164 → 8080 [ACK] Seq=1367 Ack=33302 Win=94848 Len=0 TSval=462896721 TSecr=462896720
17	0.035504	127.0.0.1	127.0.0.1	TCP	8258	8080 → 40164 [PSH, ACK] Seq=33302 Ack=1367 Win=64128 Len=8192 TSval=462896721 TSecr=462896721
18	0.036081	127.0.0.1	127.0.0.1	HTTP	712	GET /image2.jpg HTTP/1.1
19	0.036112	127.0.0.1	127.0.0.1	TCP	66	8080 → 40180 [ACK] Seq=1 Ack=647 Win=64896 Len=0 TSval=462896722 TSecr=462896722
20	0.036805	127.0.0.1	127.0.0.1	TCP	47490	8080 → 40164 [ACK] Seq=41494 Ack=1367 Win=64128 Len=47424 TSval=462896723 TSecr=462896721

Figura 1 - Captura de tráfego evidenciando requisições persistentes e multiplexação de portas na conexão TCP.

4. Conclusões

Os resultados obtidos na avaliação experimental confirmam que o sistema cumpre integralmente os objetivos estipulados para a primeira versão do Produto Mínimo Viável (MVP 1). O uso da biblioteca *Pthreads* demonstrou-se eficaz e de baixo custo computacional para a resolução da concorrência, enquanto o manuseio adequado dos descritores e cabeçalhos garantiu o funcionamento do *Keep-Alive*.

Com a infraestrutura base estabelecida, as próximas etapas do projeto (MVP 2) concentrar-se-ão na expansão do código para suportar requisitos não-funcionais mais complexos. Os desenvolvimentos futuros incluirão a medição do *Round-Trip-Time* (RTT) para estimativa de atraso fim-a-fim e largura de banda, a implementação de um controle de admissão de clientes pautado na vazão máxima do servidor e a introdução de uma

tabela de controle de taxas (QoS) baseada no endereço IP de origem das requisições.

5. Referências

KUROSE, James F.; ROSS, Keith W. *Redes de Computadores e a Internet: Uma Abordagem Top-Down*. 6. ed. São Paulo: Pearson Education do Brasil, 2013.

FIELDING, R. et al. *Hypertext Transfer Protocol -- HTTP/1.1*. RFC 2616, Network Working Group, 1999.